

# Sarthi Platform: Common Technical Considerations & Edge Cases

## Overview

This document outlines critical technical considerations, edge cases, and quirks that apply across the entire Sarthi platform. These issues must be addressed in every portal and component to ensure reliable, HIPAA-compliant operation.

## 1. Time Zone & DateTime Handling

### 1.1 Multi-System Time Zone Synchronization

**The Core Problem:**

- **Sarthi Platform:** Runs in browser timezone (patient's local time)
- **EMR Systems:** Often use practice/facility timezone
- **External Labs:** May use their local timezone
- **Insurance APIs:** Could be in any timezone
- **Cloud Functions:** Default to UTC

**Implementation Requirements:**

typescript

```

interface TimestampStandard {
  // ALWAYS store in UTC
  storedValue: string; // ISO 8601 with 'Z' suffix: "2025-08-20T14:30:00.000Z"

  // ALWAYS include timezone context
  originalTimezone: string; // IANA timezone: "America/New_York"
  facilityTimezone: string; // Practice timezone: "America/Phoenix"

  // Display helpers
  displayInUserTimezone: string;
  displayInFacilityTimezone: string;
}

// Required utility functions for EVERY component
const createTimestamp = (localDateTime: Date, timezone: string): TimestampStandard => {
  return {
    storedValue: localDateTime.toISOString(),
    originalTimezone: timezone,
    facilityTimezone: getFacilityTimezone(),
    displayInUserTimezone: formatInTimezone(localDateTime, getUserTimezone()),
    displayInFacilityTimezone: formatInTimezone(localDateTime, getFacilityTimezone())
  };
};

```

## Specific Edge Cases:

1. **Appointment Scheduling:** Patient in EST books with practice in MST
2. **Lab Result Timestamps:** Lab in PST, patient in CST, practice in EST
3. **Prescription Times:** "Take at 8 AM" - whose 8 AM?
4. **Audit Logs:** HIPAA requires accurate timestamps across jurisdictions
5. **Daylight Saving Transitions:** Appointments during 2 AM hour that doesn't exist/repeats

## Critical Rules:

typescript

```
// NEVER do this
const badTimestamp = new Date().toString(); // "Mon Aug 20 2025 14:30:00 GMT-0700"

// ALWAYS do this
const goodTimestamp = new Date().toISOString(); // "2025-08-20T21:30:00.000Z"

// For display, ALWAYS convert from UTC
const displayTime = new Intl.DateTimeFormat('en-US', {
  timeZone: userTimezone,
  dateStyle: 'medium',
  timeStyle: 'short'
}).format(new Date(storedUTCTimestamp));
```

## 1.2 EMR Integration Time Sync

### FHIR DateTime Handling:

```
typescript

interface FHIRTimestampSync {
  // EMR sends: "2025-08-20T14:30:00-07:00" (practice timezone)
  emrTimestamp: string;

  // Convert to Sarthi standard (UTC)
  sarthiTimestamp: string; // "2025-08-20T21:30:00.000Z"

  // Validation
  isValidFHIREDateime: (timestamp: string) => boolean;
  convertFromEMRTIMEZONE: (emrTime: string, emrTimezone: string) => string;
}
```

## 2. Data Synchronization & Conflict Resolution

### 2.1 Cross-Portal State Management

#### The Problem:

Multiple users can modify the same patient data simultaneously across different portals.

#### Race Condition Scenarios:

```
typescript
```

```
interface DataConflictScenario {
  scenario1: "MA updates patient status while Doctor opens chart";
  scenario2: "Billing modifies claim while Doctor submits new claim";
  scenario3: "Front office reschedules while patient books online";
  scenario4: "Multiple staff accessing same lab result simultaneously";
}
```

## Conflict Resolution Strategy:

typescript

```
interface ConflictResolution {
  optimisticLocking: {
    lastModified: string; // ISO timestamp
    version: number; // Increments with each change
    modifiedBy: string; // User ID
  };

  conflictDetection: {
    checkVersionOnSave: boolean;
    compareTimestamps: boolean;
    requireUserResolution: boolean;
  };

  resolutionStrategy: 'last_write_wins' | 'user_merge' | 'automatic_merge' | 'escalate_to_admin';
}

// Implementation pattern for ALL data modifications
const updatePatientData = async (patientId: string, updates: any, currentVersion: number) => {
  try {
    const result = await updateWithOptimisticLock(patientId, updates, currentVersion);
    return result;
  } catch (ConflictError) {
    // Show user the conflict and let them resolve
    const currentData = await fetchLatestPatientData(patientId);
    const resolution = await showConflictResolutionDialog(updates, currentData);
    return await updatePatientData(patientId, resolution, currentData.version);
  }
};
```

## 2.2 Browser Tab Synchronization

### Multiple Tab Issues:

typescript

```
interface MultiTabSync {
  problems: [
    "Patient opens portal in 2 tabs, books appointment in both",
    "Staff member has billing portal open while patient pays bill online",
    "Doctor reviews lab in one tab while technician updates in another"
  ];

  solution: "BroadcastChannel API + localStorage events";
}

// Required implementation for ALL portals
class TabSynchronizer {
  private broadcast: BroadcastChannel;

  constructor(channelName: string) {
    this.broadcast = new BroadcastChannel(channelName);
    this.broadcast.onmessage = this.handleCrossTabUpdate;
    window.addEventListener('storage', this.handleStorageUpdate);
  }

  notifyOtherTabs(action: string, data: any) {
    this.broadcast.postMessage({ action, data, timestamp: Date.now() });
  }

  handleCrossTabUpdate = (event: MessageEvent) => {
    // Update local state based on changes from other tabs
    this.updateLocalState(event.data);
  };
}
```

---

## 3. Patient Identity & Data Consistency

### 3.1 Patient Matching Across Systems

#### Identity Resolution Problems:

typescript

```
interface PatientIdentityIssues {
  duplicateRecords: "John Smith vs Jonathan Smith - same person?";
  namingVariations: "Maria Garcia-Lopez vs Maria Lopez vs Maria G. Lopez";
  demographicChanges: "Patient moves, changes phone, gets married";
  insuranceChanges: "New policy number, different member ID";
  minorVsAdult: "Pediatric patient turns 18, needs adult record";
}
```

## Master Patient Index (MPI) Strategy:

typescript

```
interface PatientMasterRecord {
  sarthiPatientId: string; // Internal UUID - never changes
  emrPatientId?: string; // External EMR ID
  alternatelds: {
    mrnNumber?: string;
    ssnLast4?: string;
    dateOfBirth: string; // YYYY-MM-DD
    insuranceMemberId?: string;
  };

  // Demographic data that can change
  demographics: {
    names: PatientName[]; // Historical names
    addresses: PatientAddress[]; // Historical addresses
    phoneNumbers: PhoneNumber[]; // Historical numbers
    emailAddresses: EmailAddress[]; // Historical emails
  };

  // Confidence scoring for matching
  matchingConfidence: number; // 0-100
  lastVerificationDate: string;
  verificationMethod: 'manual' | 'automated' | 'patient_confirmed';
}
```

## 3.2 Data Integrity Validation

### Cross-System Validation Rules:

typescript

```
interface DataValidationRules {  
  patientAge: {  
    rule: "Age calculated from DOB must match across all systems";  
    validation: (dob: string, claimedAge: number) => boolean;  
  };  
  
  insuranceEligibility: {  
    rule: "Insurance status must be verified before service";  
    validation: (memberId: string, serviceDate: string) => Promise<boolean>;  
  };  
  
  providerLicensing: {  
    rule: "Provider must be licensed in patient's state";  
    validation: (providerId: string, patientState: string) => boolean;  
  };  
  
  appointmentLogic: {  
    rule: "Appointment duration must match service type";  
    validation: (serviceType: string, duration: number) => boolean;  
  };  
}
```

---

## 4. Network & Connectivity Issues

### 4.1 Offline/Poor Connectivity Handling

#### Offline-First Architecture:

typescript

```
interface OfflineStrategy {  
  criticalData: "Cache patient data, appointments, urgent messages";  
  localChanges: "Queue modifications for sync when online";  
  conflictResolution: "Merge local changes with server updates";  
  userFeedback: "Clear indicators of offline status and sync status";  
}
```

```
// Required for ALL components that modify data
```

```
class OfflineManager {  
  private changeQueue: QueuedChange[] = [];  
  private isOnline: boolean = navigator.onLine;  
  
  constructor() {  
    window.addEventListener('online', this.syncPendingChanges);  
    window.addEventListener('offline', this.handleOfflineMode);  
  }  
  
  queueChange(change: QueuedChange) {  
    this.changeQueue.push({  
      ...change,  
      timestamp: Date.now(),  
      id: crypto.randomUUID()  
    });  
  
    if (this.isOnline) {  
      this.syncPendingChanges();  
    }  
  }  
  
  async syncPendingChanges() {  
    for (const change of this.changeQueue) {  
      try {  
        await this.applyChange(change);  
        this.removeFromQueue(change.id);  
      } catch (error) {  
        // Handle conflicts, retry logic, etc.  
        this.handleSyncError(change, error);  
      }  
    }  
  }  
}
```



## 4.2 API Rate Limiting & Throttling

### External Service Limits:

typescript

```
interface ExternalServiceLimits {
  insuranceVerification: {
    rateLimit: "10 requests per minute per practice";
    strategy: "Queue requests, batch when possible";
    fallback: "Use cached data if available";
  };

  labResultFetching: {
    rateLimit: "100 requests per hour";
    strategy: "Prioritize urgent results, batch routine";
    fallback: "Manual entry workflow";
  };

  prescriptionSubmission: {
    rateLimit: "5 requests per minute per provider";
    strategy: "Queue with priority (urgent meds first)";
    fallback: "Paper prescription generation";
  };
}

// Required rate limiting implementation
class RateLimiter {
  private requestQueue: Map<string, Request[]> = new Map();
  private limits: Map<string, RateLimit> = new Map();

  async makeRequest(service: string, request: Request): Promise<Response> {
    const limit = this.limits.get(service);
    const queue = this.requestQueue.get(service) || [];

    if (this.exceedsLimit(service, queue.length)) {
      // Queue request for later
      return this.queueRequest(service, request);
    }

    return this.executeRequest(request);
  }
}
```

---

## 5. Security & Compliance Edge Cases

### 5.1 Session Management Across Portals

#### Multi-Portal Session Issues:

```
typescript

interface SessionComplexity {
  problems: [
    "User logged into Patient Portal, opens Clinician Portal link",
    "Session expires in one portal but not others",
    "Different auth levels for different portals (patient vs staff)",
    "Remember me in patient portal but not clinician portal"
  ];

  solution: "Unified session management with role-based access";
}

interface UnifiedSession {
  sessionId: string;
  userId: string;
  roles: UserRole[]; // ['patient', 'ma', 'doctor', 'billing']
  portalAccess: {
    [portalId: string]: {
      hasAccess: boolean;
      lastAccessed: string;
      expiresAt: string;
    };
  };
  securityLevel: 'basic' | 'elevated' | 'admin';
  mfaVerified: boolean;
  ipAddress: string;
  deviceFingerprint: string;
}
```

### 5.2 PHI Handling in Browser Storage

#### Browser Storage Security:

```
typescript
```

```

interface PHISStorageRules {
  localStorage: {
    allowed: "Session IDs, user preferences, non-PHI cache";
    forbidden: "Patient names, DOB, SSN, diagnosis codes, lab results";
    encryption: "Required for any healthcare-related data";
  };

  sessionStorage: {
    allowed: "Temporary workflow data, form drafts";
    forbidden: "PHI that persists beyond session";
    clearOnPageUnload: true;
  };

  indexedDB: {
    allowed: "Encrypted cache for offline functionality";
    encryption: "AES-256 with user-derived keys";
    autoExpiry: "24 hours maximum";
  };
}

// Required encryption for ALL PHI storage
class PHISStorage {
  private static async encrypt(data: any, key: CryptoKey): Promise<string> {
    const iv = crypto.getRandomValues(new Uint8Array(16));
    const encoded = new TextEncoder().encode(JSON.stringify(data));
    const encrypted = await crypto.subtle.encrypt(
      { name: 'AES-GCM', iv },
      key,
      encoded
    );
    return btoa(String.fromCharCode(...new Uint8Array(iv), ...new Uint8Array(encrypted)));
  }

  static async storePHI(key: string, data: any): Promise<void> {
    const cryptoKey = await this.getUserKey();
    const encrypted = await this.encrypt(data, cryptoKey);
    sessionStorage.setItem(key, encrypted);

    // Auto-expire
    setTimeout(() => {
      sessionStorage.removeItem(key);
    }, 24 * 60 * 60 * 1000); // 24 hours
  }
}

```

```
}
```

```
}
```

---

## 6. Browser Compatibility & Performance

### 6.1 Browser-Specific Issues

#### Known Browser Quirks:

typescript

```

interface BrowserQuirks {
  safari: {
    issue: "localStorage disabled in private browsing";
    workaround: "Detect and fallback to memory storage";
    impact: "Patient data won't persist across tabs";
  };

  mobileSafari: {
    issue: "Date input format differences";
    workaround: "Custom date picker with explicit formatting";
    impact: "Appointment booking fails";
  };

  chrome: {
    issue: "Aggressive memory management kills service workers";
    workaround: "Periodic keepalive messages";
    impact: "Offline functionality breaks";
  };

  firefox: {
    issue: "File upload limitations in certain versions";
    workaround: "Progressive enhancement with feature detection";
    impact: "Document upload fails";
  };

  ie11: {
    issue: "No native Promise support, limited ES6";
    workaround: "Polyfills and transpilation";
    impact: "Complete application failure";
  };
}

```

*// Required feature detection*

```

class BrowserCompatibility {
  static detectCapabilities(): BrowserCapabilities {
    return {
      hasLocalStorage: this.testLocalStorage(),
      hasServiceWorker: 'serviceWorker' in navigator,
      hasWebCrypto: 'crypto' in window && 'subtle' in crypto,
      hasFileAPI: 'File' in window && 'FileReader' in window,
      hasWebSocket: 'WebSocket' in window,
      hasBroadcastChannel: 'BroadcastChannel' in window
    };
  }
}

```

```
}

static testLocalStorage(): boolean {
  try {
    localStorage.setItem('test', 'test');
    localStorage.removeItem('test');
    return true;
  } catch {
    return false;
  }
}
```

## 6.2 Performance Edge Cases

### Memory Management:

```
typescript
```

```

interface MemoryManagement {
  imageUploads: {
    issue: "Large medical images (X-rays, etc.) crash browser";
    solution: "Client-side compression before upload";
    maxSize: "10MB after compression";
  };

  longRunningPages: {
    issue: "Memory leaks in dashboards left open all day";
    solution: "Periodic cleanup, component unmounting";
    monitoring: "Track memory usage, warn at 80% limit";
  };

  realtimeUpdates: {
    issue: "WebSocket connections accumulate without cleanup";
    solution: "Connection pooling, automatic reconnection";
    fallback: "Polling when WebSocket fails";
  };
}

```

*// Required cleanup patterns*

```

class ResourceManager {
  private static activeConnections: Set<WebSocket> = new Set();
  private static activeTimers: Set<number> = new Set();

  static addConnection(ws: WebSocket) {
    this.activeConnections.add(ws);
    ws.addEventListener('close', () => {
      this.activeConnections.delete(ws);
    });
  }

  static cleanup() {
    // Close all connections
    this.activeConnections.forEach(ws => ws.close());

    // Clear all timers
    this.activeTimers.forEach(timer => clearTimeout(timer));

    // Force garbage collection if available
    if ('gc' in window) {
      (window as any).gc();
    }
  }
}

```

```
}
```

```
}
```

---

## 7. Integration Edge Cases

### 7.1 EMR Integration Failures

#### Fallback Strategies:

```
typescript
```



```

interface EMRFailureHandling {
  patientDataSync: {
    failure: "EMR API is down";
    fallback: "Use cached patient data with staleness warning";
    userNotification: "EMR connection issue - using last known data";
  };

  appointmentSync: {
    failure: "Can't sync new appointment to EMR";
    fallback: "Queue appointment for manual entry";
    userNotification: "Appointment created - pending EMR sync";
  };

  prescriptionSubmission: {
    failure: "ePrescribe system unavailable";
    fallback: "Generate paper prescription";
    userNotification: "Electronic prescribing unavailable - printing paper Rx";
  };
}

```

*// Required for ALL external integrations*

```

class IntegrationResilience {
  private static retryConfig = {
    maxRetries: 3,
    backoffMultiplier: 2,
    initialDelay: 1000
  };

  static async callWithFallback<T>(
    primaryCall: () => Promise<T>,
    fallbackCall: () => Promise<T>,
    userNotification: string
  ): Promise<T> {
    try {
      return await this.retryWithBackoff(primaryCall);
    } catch (primaryError) {
      console.warn('Primary integration failed:', primaryError);

      // Notify user of degraded functionality
      this.showUserNotification(userNotification);

      try {
        return await fallbackCall();
      }
    }
  }
}

```

```
    } catch (fallbackError) {  
      // Both failed - this is critical  
      throw new CriticalIntegrationError(primaryError, fallbackError);  
    }  
  }  
}  
}
```

## 8. Implementation Guidelines

### 8.1 Required Error Handling Patterns

#### Standard Error Handling:

```
typescript  
  
// EVERY async operation must follow this pattern  
const handleAsyncOperation = async (operation: () => Promise<any>) => {  
  try {  
    const result = await operation();  
    return { success: true, data: result };  
  } catch (error) {  
    // Log for debugging  
    console.error('Operation failed:', error);  
  
    // Log for compliance (if PHI involved)  
    if (isPHIOperation(operation)) {  
      auditLog.error('PHI operation failed', { error, user: currentUser });  
    }  
  
    // User-friendly error message  
    const userMessage = getUserFriendlyError(error);  
    showErrorNotification(userMessage);  
  
    return { success: false, error: userMessage };  
  }  
};
```

### 8.2 Testing Requirements

#### Edge Case Testing:

typescript

```
interface RequiredEdgeCaseTests {  
  timezoneTests: [  
    "Appointment scheduling across timezones",  
    "Daylight saving time transitions",  
    "Lab results from different timezones"  
  ];  
  
  connectivityTests: [  
    "Offline operation and sync",  
    "Slow network conditions",  
    "Intermittent connectivity"  
  ];  
  
  concurrencyTests: [  
    "Multiple users editing same record",  
    "Multiple browser tabs open",  
    "Session expiry during operation"  
  ];  
  
  dataIntegrityTests: [  
    "Corrupted localStorage recovery",  
    "Invalid data format handling",  
    "Missing required fields"  
  ];  
}
```

## 8.3 Monitoring & Alerting

### Required Monitoring:

typescript

```
interface MonitoringRequirements {  
  performanceMetrics: {  
    pageLoadTime: "< 2 seconds 95th percentile";  
    apiResponseTime: "< 500ms average";  
    errorRate: "< 1% of all operations";  
  };  
  
  businessMetrics: {  
    sessionExpiryRate: "Track unexpected logouts";  
    dataLossIncidents: "Any data not properly synced";  
    integrationFailures: "EMR/external service failures";  
  };  
  
  securityMetrics: {  
    failedLogins: "Track brute force attempts";  
    sessionHijacking: "Detect IP/device changes";  
    dataAccess: "All PHI access must be logged";  
  };  
}
```

This comprehensive document covers the critical edge cases and cross-cutting concerns that your development team must address to build a robust, compliant healthcare platform. These considerations should be referenced during the implementation of every feature across all portals.